

**Diseño de pruebas estructurales del Sistema de Gestión y Reporte de Fallas en Alumbrado
Público de Medellín**

Santiago Restrepo Torres

Institución Universitaria Pascual Bravo

Ingeniería de Software II

Juan Camilo Palacio Alcaraz

Octubre de 2026

Introducción

En este trabajo analizamos cómo se registrará, priorizará y revisará el cierre de un ticket del SGF-AP. La aplicación Android se desarrollará en Python y accederá mediante una API a la base de datos actual y a sus acciones. Usaremos un modelo algorítmico de referencia en Python que permita ejecutar las operaciones del plan. Este modelo nos servirá para revisar las instrucciones y las pruebas, sin dar por definida la tecnología de la API.

En registrar nos interesa seguir `ticket_id` desde el resultado de la inserción hasta el retorno del método. También revisaremos la variable `i`, porque debe avanzar por todas las evidencias sin repetirlas ni omitirlas. La asignación de `self.id` deberá ocurrir después de confirmar los cambios. Si una operación falla, el recorrido cambiará hacia el manejo de la excepción; por eso incluiremos ese recorrido en el grafo.

En priorizar y revisar_cierre seguiremos las variables obtenidas de las consultas: estado, prioridad, solución y conteo de evidencias. Prepararemos tickets existentes, catálogos cargados y resultados con los tipos que esperan los métodos. Relacionaremos cada condición con sus entradas, las llamadas previstas y el resultado que comprobaremos. Los diagramas, el código de referencia y los casos deberán describir las mismas operaciones sobre Ticket.

El alcance será el flujo de los tres métodos propuestos de Ticket, con sus condiciones, retornos y manejo de excepciones. La interfaz conservará las opciones correspondientes a cada perfil. Los resultados del modelo de referencia se identificarán por separado de las pruebas posteriores de la aplicación y de la API.

Historias de usuario y objetivos internos de validación

Trabajaremos con HU-001, HU-005 y HU-010. En HU-001 revisaremos los datos del reporte, la dirección, la asociación de evidencias y el registro inicial de RF-001, RF-002, RF-004 y RF-005. En HU-005 usaremos la clasificación existente para comprobar la prioridad de RF-006 y RF-007, junto con el historial de RF-021. En HU-010 revisaremos las condiciones de cierre de RF-018 y el evento de RF-021. Cada prueba se limitará a la parte del requisito que aparece en los campos y las operaciones del modelo.

Las entradas de cada caso mantendrán válidos los datos que no estemos probando. Por ejemplo, al revisar `address_id=None` usaremos una descripción con contenido, para que el método alcance esa segunda condición. Prepararemos los estados y acciones con sus nombres actuales y obtendremos sus identificadores de las filas de prueba. En `priorizar` y `revisar_cierre` usaremos tickets existentes y resultados de consulta con la estructura que recibe el modelo; estas serán condiciones de preparación de las pruebas.

Para los tres métodos probaremos por separado cada llamada que pueda producir una excepción. Cuando `rollback` termine correctamente, comprobaremos que se propague la misma excepción recibida. Si `rollback` también falla, comprobaremos el error de reversión y que conserve el error anterior en su contexto. En ninguno de esos recorridos se deberá devolver un resultado de éxito.

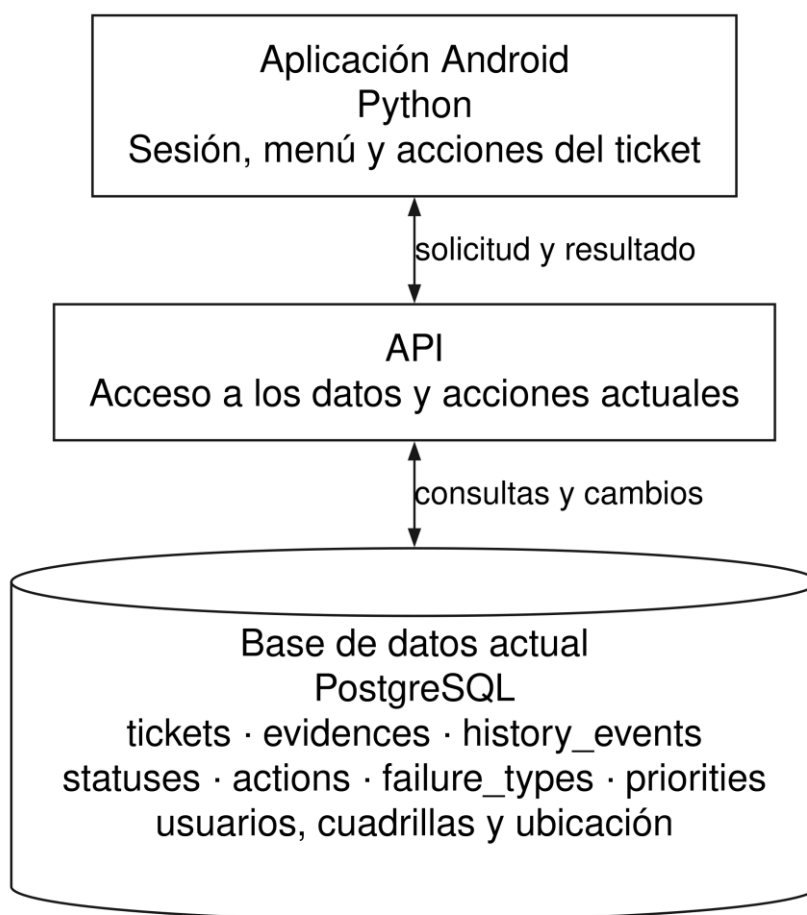
Métodos, variables y condiciones de las historias seleccionadas

Historia y método	Variables y decisiones	Comprobación interna
HU-001 registrar(user_id: int) → int None	description.strip() == ""; address_id is None; ticket_id; self.id; i < len(self.evidencias).	Revisaremos las dos guardas y el ciclo con cero, una y varias evidencias. Los INSERT de evidencias e historial usarán el ticket_id recuperado. self.id solo se asignará después de commit.
HU-005 priorizar(user_id: int) → str None	estado, prioridad_id y prioridad salen del SELECT. La condición será estado != "Validated".	Comprobaremos el id enviado al UPDATE y el nombre retornado. High y Standard serán datos de la misma ruta. El otro estado terminará sin escrituras, con commit de lectura.
HU-010 revisar_cierre(user_id: int, action: str) → str	estado != "PendingClosure"; action == "Approval"; solution is None; solution.strip() == ""; cantidad == 0.	Separaremos cada condición y la asignación de destino. Revisaremos los argumentos del UPDATE y del historial, los retornos anticipados y los errores de todas las llamadas.

Diseño de componentes

La aplicación Android en Python organizará la sesión y las opciones de cada perfil para trabajar con Ticket. Mediante la API se conectará con la base de datos actual y con las acciones que esta maneja. El diagrama muestra esa conexión y las dependencias que intervienen en las tres historias seleccionadas.

Componentes de la aplicación y acceso a los datos



Modelo de clases

Ticket y las firmas del modelo de referencia

Ticket	
Atributos correspondientes a tickets	
id: int None code: str reported_by: int None failure_type_id: int None description: str address_id: int None status_id: int None	priority_id: int None contact_email: str None contact_phone: str None diagnosis: str None solution: str None crew_id: int None created_at: Any
Relación con evidences	
evidencias: list[dict[str, Any]]	
Recursos de ejecución; no son columnas	
conexion: Any cursor: Any	
Operaciones	
+ registrar(user_id: int) → int None + priorizar(user_id: int) → str None + revisar_cierre(user_id: int, action: str) → str	

Flujo de datos y preparación de las variables

Prepararemos una conexión y un cursor antes de llamar los métodos del modelo. Los atributos tendrán tipos compatibles con el esquema y las consultas devolverán las columnas esperadas. El ticket consultado existirá y sus referencias serán válidas. Estas condiciones nos permitirán cambiar una entrada o provocar un fallo a la vez, de modo que una validación anterior no oculte la ruta que queremos revisar.

El código del ticket llegará generado, sin fijar un formato nuevo. La dirección seleccionada llegará en `address_id`. En la revisión, `action` procederá de las opciones Approval y Rejection del coordinador. La solución se tomará de la fila consultada. El conteo se filtrará por el `id` del ticket; la suficiencia de las fotografías será parte de la revisión del coordinador.

Tipos, valores y límites que intervienen en las pruebas

Variable o campo	Condición precisa	Datos de prueba y alcance
<code>description: str</code>	<code>description.strip() == ""</code>	Probaremos una descripción vacía, una que solo contenga espacios, tabulaciones o saltos de línea, y una descripción con contenido, como “Luz apagada”. La guarda se evalúa antes de SQL. No agregamos un máximo a TEXT.
<code>address_id: int None</code>	<code>address_id is None</code>	None y una dirección existente. La descripción tendrá contenido. SQL admite NULL; la guarda pertenece al registro de HU-001.
<code>evidencias: list[dict]</code>	<code>i < len(self.evidencias)</code>	0, 1 y 3 entradas para revisar cero vueltas, una vuelta y varias. Cada entrada tendrá <code>file_path</code> y <code>captured_at</code> . Son cantidades de prueba.
<code>ticket_id: int; i: int</code>	<code>ticket_id</code> se obtiene con <code>fetchone()[0]</code> ; <code>i</code> inicia en 0.	<code>id=101</code> simulado; para tres evidencias, <code>i=0, 1, 2</code> y 3. Los INSERT de evidencias e historial conservarán el <code>ticket_id</code> y el <code>user_id</code> de la sesión.
<code>estado: str;</code> <code>prioridad_id: int;</code> <code>prioridad: str</code>	<code>estado != "Validated"</code>	("Validated", 97, "High") y ("Registered", 98, "Standard"). Los números son ids simulados.
<code>solution: str None</code>	<code>solution is None; solution.strip() == ""</code>	None, "", " " y "Fotocelda reemplazada". La normalización de None tiene una rama propia.

Variable o campo	Condición precisa	Datos de prueba y alcance
action: str; cantidad: int	action == "Approval"; cantidad == 0	Approval y Rejection, dominio preparado desde las opciones del coordinador. COUNT devuelve 0 o un entero positivo para el ticket seleccionado.
code: str; file_path: str	tickets.code: VARCHAR(24), UNIQUE; evidences.file_path: TEXT, UNIQUE.	Los 24/25 caracteres y valores repetidos son pruebas complementarias de integración SQL. No añaden condiciones if ni caminos independientes al modelo.

Cargaremos statuses y actions por sus nombres del SQL antes de las pruebas de integración, porque los scripts definen los valores permitidos. Los métodos resolverán sus identificadores mediante consultas. Para priorizar se seguirá la relación tickets.failure_type_id → failure_types.priority_id → priorities.id.

Datos actuales del catálogo usados por priorizar

failure_types.name	priorities.name
Exposed cable	High
Pole fall risk	High
Box without cover	High
Light out	Standard
Intermittent	Standard
Tree Branches	High

Algoritmos, grafos y complejidad ciclomática

Presentaremos el código de referencia y su grafo para cada método. En los grafos mostraremos las condiciones, las consultas, las escrituras y las rutas que llevan al bloque except. También seguiremos las variables desde su inicialización hasta su uso en las operaciones y en el retorno. Por ejemplo, revisaremos cómo el identificador obtenido al insertar el ticket se conserva en `ticket_id` y se utiliza para asociar sus evidencias y su historial.

En el registro, `commit()` confirmará las inserciones del ticket, sus evidencias y el evento de historial dentro de una misma transacción. Lo ejecutaremos después de completar esas operaciones, para que se confirmen juntas. Solo después de una confirmación correcta asignaremos el identificador a `self.id` y lo devolveremos como resultado. En la priorización y el cierre, `commit()` confirmará la actualización del ticket junto con su evento de historial. En los recorridos que terminen después de consultar datos, sin realizar escrituras, lo utilizaremos para finalizar la transacción de lectura.

Si una operación falla, el bloque except intentará ejecutar `rollback()` para deshacer los cambios que todavía estén pendientes en esa transacción. Por ejemplo, si se inserta el ticket pero falla la inserción de su historial, una reversión correcta deshará la inserción del ticket y las evidencias que se hayan agregado durante ese intento. El grafo mostrará tanto ese recorrido como el posible fallo del propio `rollback()`. Cuando la reversión termine correctamente, comprobaremos que se propague la excepción original. Si también falla la reversión, comprobaremos que se propague ese segundo error y que conserve el anterior como contexto.

Calcularemos $V(G) = E - N + 2$ sobre cada grafo conectado completo, con E como cantidad de aristas y N como cantidad de nodos. La matriz relacionará los recorridos con los casos que los ejecutarán. Para cada arista indicaremos qué caso permite comprobarla, incluidas

las salidas por error. El ciclo de evidencias tendrá además casos con distinta cantidad de entradas para revisar su avance.

En cada caso compararemos el retorno o la excepción, los argumentos enviados a las consultas y la secuencia de llamadas. Un fallo en una escritura deberá impedir las operaciones posteriores. Una falla durante commit tendrá una llamada de confirmación intentada, seguida del manejo del error; no se contará como una ruta sin llamada a commit.

Cada grafo tendrá una entrada y una salida común que reunirá los retornos y las propagaciones de error. Incluiremos una arista por el resultado correcto de cada execute, fetchone, commit y rollback, y otra por su fallo. Por eso el cálculo corresponde al grafo con excepciones dibujado en este documento. Las operaciones sobre los tipos preparados no incorporarán fallos de memoria o interrupciones del proceso.

Cálculo sobre los grafos completos

Método	N	E	P	$V(G) = E - N + 2P$	Base
registrar	17	25	1	$25 - 17 + 2 = 10$	PE-01–PE-10
priorizar	14	21	1	$21 - 14 + 2 = 9$	PE-11–PE-19
revisar_cierre	27	42	1	$42 - 27 + 2 = 17$	PE-20–PE-36

Trabajaremos con 36 caminos independientes: 10 de registro, 9 de priorización y 17 de revisión de cierre. Cada camino posterior de una base incorporará una arista que no aparece en los anteriores. Las tablas mostrarán esa arista y el recorrido completo. Esto justifica la base del grafo; las repeticiones adicionales del ciclo y las combinaciones de datos se comprobarán con casos complementarios.

Preparación común de los casos

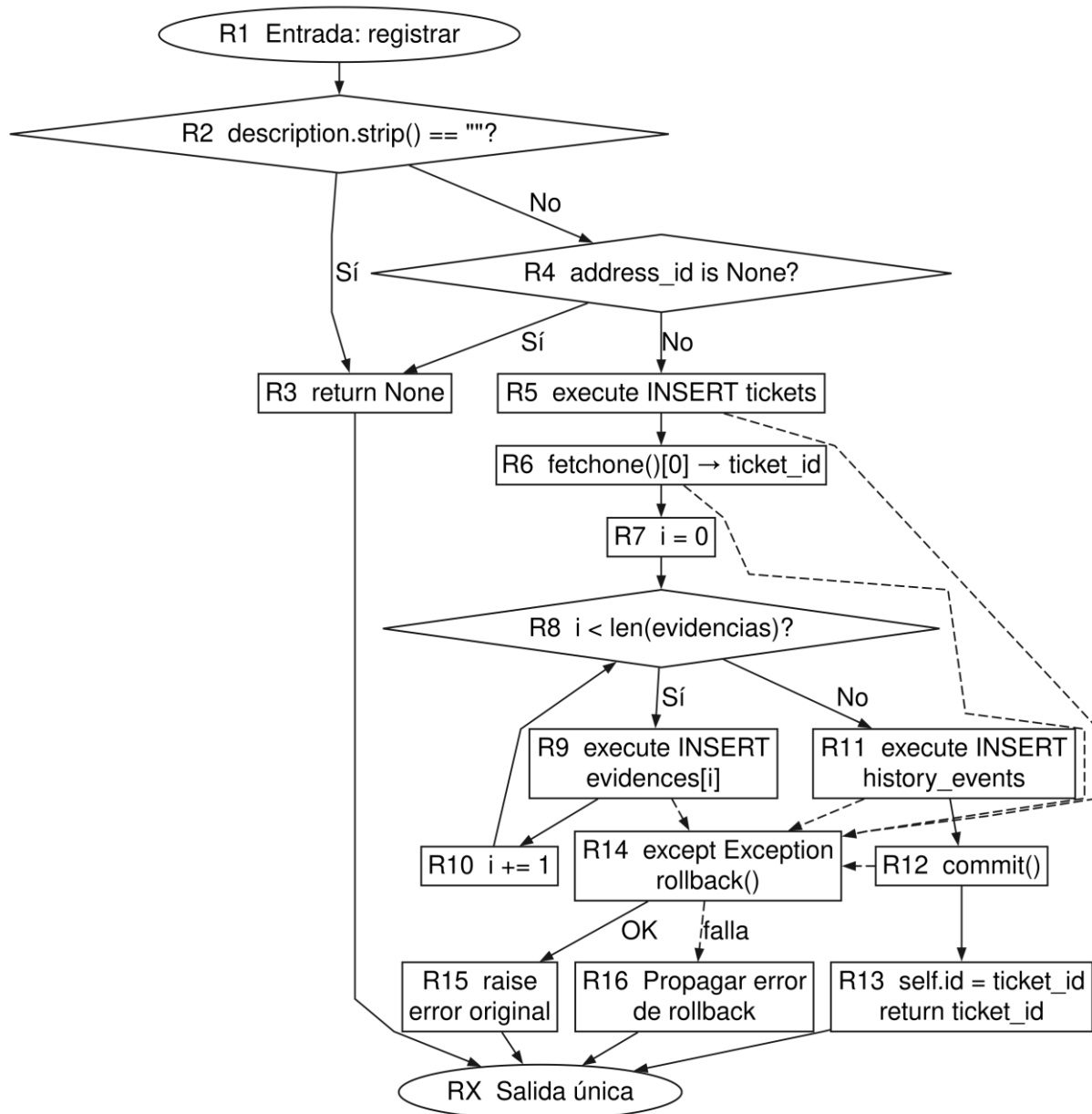
Datos simulados de partida

Base	Valores
F-R	Ticket nuevo con self.id=None, code disponible, description="Luz apagada", address_id y failure_type_id existentes; user_id=7. fetchone estará configurado para devolver (101,) si la llamada se completa. evidencias=[] salvo que el caso indique otra cantidad.
F-P	Ticket id=301; user_id=7 del coordinador. SELECT devolverá ("Validated", 97, "High"). Se usará otro resultado solo cuando lo indique el caso.
F-C	Ticket id=301; user_id=7 del coordinador; action="Approval". SELECT devolverá ("PendingClosure", "Fotocelda reemplazada") y COUNT devolverá (1,). El coordinador habrá revisado la imagen relacionada.

Los ids serán valores simulados. error_original será una instancia de RuntimeError del doble. C y R indicarán las llamadas intentadas a commit y rollback. En los casos de fallo de commit, el error se producirá antes de su efecto.

Registro de un ticket y sus evidencias

Grafo de registrar



$$N = 17; E = 25; P = 1. V(G) = 25 - 17 + 2 = 10.$$

Caminos independientes de registro

Base de caminos y arista incorporada

Caso	Recorrido completo	Arista nueva
PE-01	R1 → R2 → R4 → R5 → R6 → R7 → R8 → R11 → R12 → R13 → RX	R1 → R2
PE-02	R1 → R2 → R3 → RX	R2 → R3
PE-03	R1 → R2 → R4 → R3 → RX	R4 → R3
PE-04	R1 → R2 → R4 → R5 → R6 → R7 → R8 → R9 → R10 → R8 → R11 → R12 → R13 → RX	R8 → R9
PE-05	R1 → R2 → R4 → R5 → R14 → R15 → RX	R5 → R14
PE-06	R1 → R2 → R4 → R5 → R6 → R14 → R15 → RX	R6 → R14
PE-07	R1 → R2 → R4 → R5 → R6 → R7 → R8 → R9 → R14 → R15 → RX	R9 → R14
PE-08	R1 → R2 → R4 → R5 → R6 → R7 → R8 → R11 → R14 → R15 → RX	R11 → R14
PE-09	R1 → R2 → R4 → R5 → R6 → R7 → R8 → R11 → R12 → R14 → R15 → RX	R12 → R14
PE-10	R1 → R2 → R4 → R5 → R14 → R16 → RX	R14 → R16

Entradas y resultados internos esperados

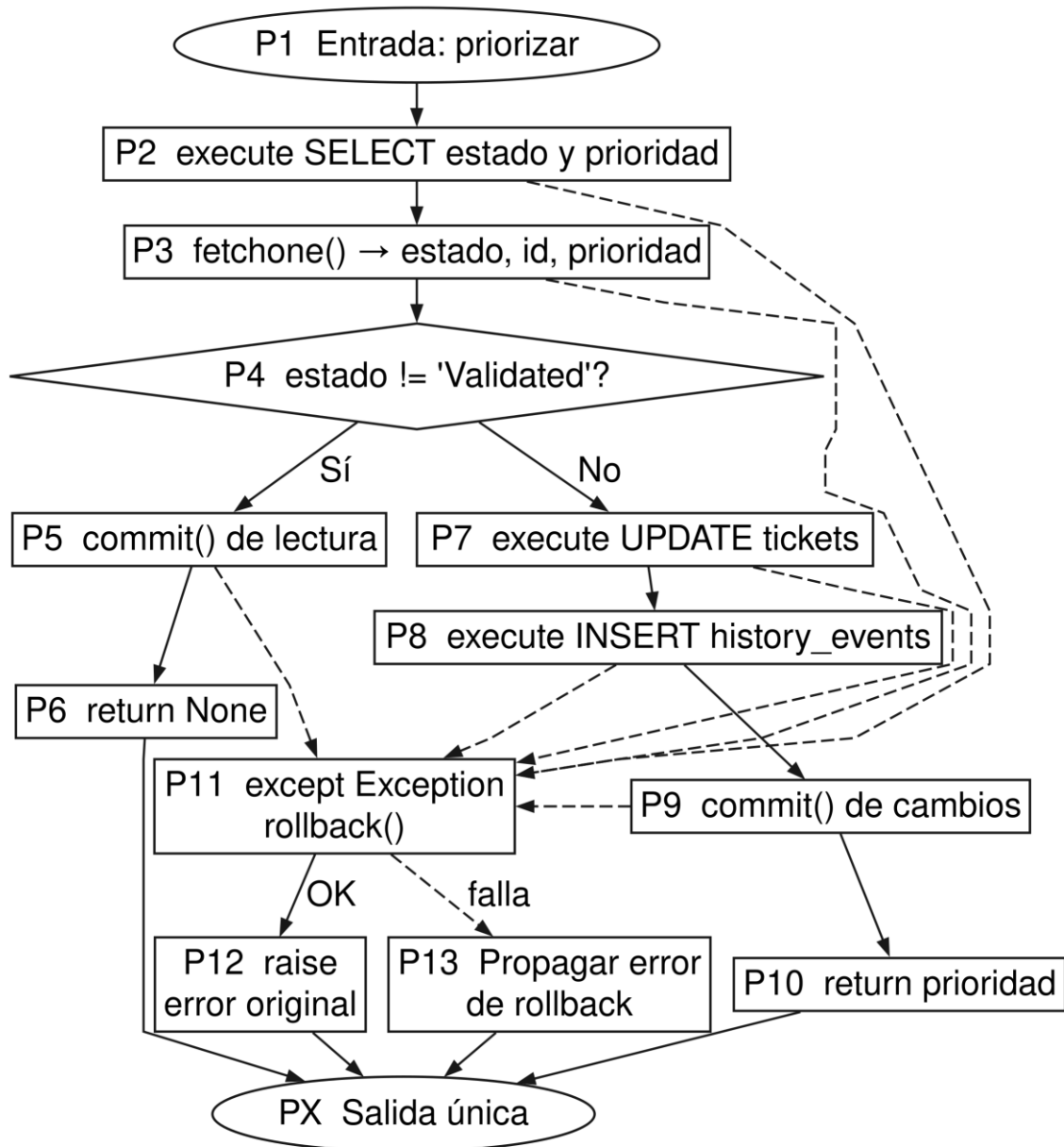
Caso	Preparación o fallo provocado	Criterio verificable
PE-01	F-R; evidencias=[].	return 101. C=1; R=0. self.id=101 después de commit; dos INSERT; ninguna inserción de evidencia.
PE-02	F-R; description="".	return None. C=0; R=0. Sin execute ni fetchone; self.id conserva None.
PE-03	F-R; address_id=None; descripción con contenido.	return None. C=0; R=0. La segunda guarda se alcanza; sin execute ni fetchone; self.id conserva None.
PE-04	F-R; una evidencia con file_path disponible y captured_at definido.	return 101. C=1; R=0. Tres INSERT; evidencia con ticket_id=101 y uploaded_by=7; i pasa de 0 a 1; self.id=101 después de commit.
PE-05	F-R; error_original en R5.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise. self.id conserva None.
PE-06	F-R; error_original en R6.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise. self.id conserva None.

Caso	Preparación o fallo provocado	Criterio verificable
PE-07	F-R con una evidencia; error_original en R9.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise. self.id conserva None.
PE-08	F-R; error_original en R11.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise. self.id conserva None.
PE-09	F-R; error_original en R12 antes del efecto de commit.	Propaga la misma instancia error_original. C=1; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise. self.id conserva None.
PE-10	F-R; error_original en R5 y error_rollback en R14.	Propaga error_rollback; __context__ is error_original. C=0; R=1. No llega al raise del manejador ni retorna éxito.

El caso recorrerá los nodos de su fila en la base anterior. Comprobaremos el orden de las llamadas y sus argumentos. Si un nodo falla, las llamadas posteriores del recorrido normal no deberán ejecutarse; el control deberá seguir por el manejador. Los conteos C y R incluyen las llamadas intentadas, aunque una de ellas lance una excepción.

Priorización individual de un ticket

Grafo de priorizar



$$N = 14; E = 21; P = 1. V(G) = 21 - 14 + 2 = 9.$$

Camino independientes de priorización

Base de caminos y arista incorporada

Caso	Recorrido completo	Arista nueva
PE-11	P1 → P2 → P3 → P4 → P7 → P8 → P9 → P10 → PX	P1 → P2
PE-12	P1 → P2 → P3 → P4 → P5 → P6 → PX	P4 → P5
PE-13	P1 → P2 → P11 → P12 → PX	P2 → P11
PE-14	P1 → P2 → P3 → P11 → P12 → PX	P3 → P11
PE-15	P1 → P2 → P3 → P4 → P5 → P11 → P12 → PX	P5 → P11
PE-16	P1 → P2 → P3 → P4 → P7 → P11 → P12 → PX	P7 → P11
PE-17	P1 → P2 → P3 → P4 → P7 → P8 → P11 → P12 → PX	P8 → P11
PE-18	P1 → P2 → P3 → P4 → P7 → P8 → P9 → P11 → P12 → PX	P9 → P11
PE-19	P1 → P2 → P11 → P13 → PX	P11 → P13

Entradas y resultados internos esperados

Caso	Preparación o fallo provocado	Criterio verificable
PE-11	F-P; lectura ("Validated", 97, "High").	return 'High'. C=1; R=0. UPDATE con (97, "Prioritized", 301); historial con (301, 7, "Prioritization").
PE-12	F-P; lectura ("Registered", 98, "Standard").	return None. C=1; R=0. SELECT y fetchone; commit de lectura; no UPDATE ni INSERT.
PE-13	F-P; error_original en P2.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-14	F-P; error_original en P3.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-15	Datos de PE-12; error_original en P5.	Propaga la misma instancia error_original. C=1; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-16	F-P; error_original en P7.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-17	F-P; error_original en P8.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.

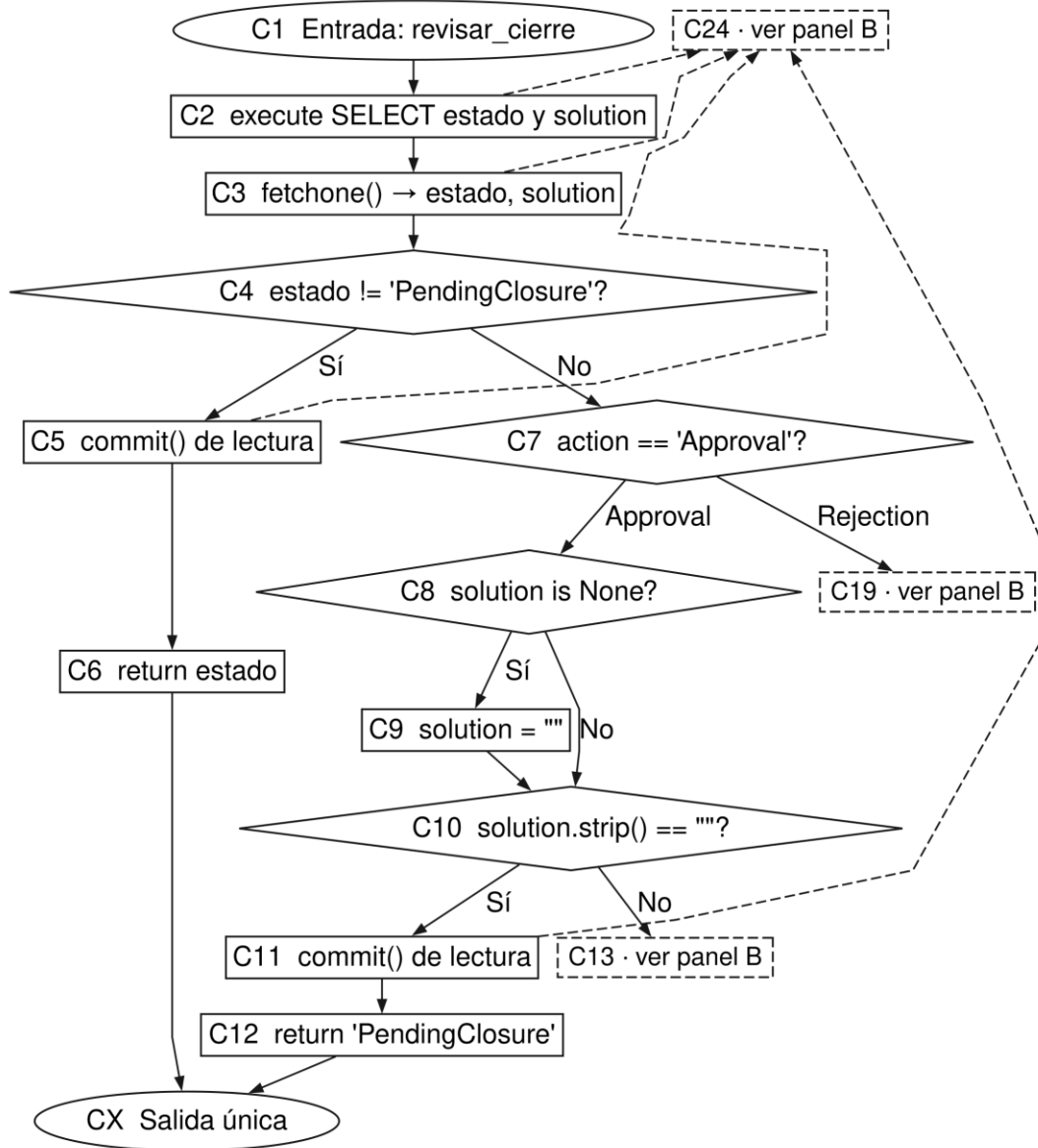
Caso	Preparación o fallo provocado	Criterio verificable
PE-18	F-P; error_original en P9 antes del efecto de commit.	Propaga la misma instancia error_original. C=1; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-19	F-P; error_original en P2 y error_rollback en P11.	Propaga error_rollback; __context__ is error_original. C=0; R=1. No llega al raise del manejador ni retorna éxito.

El caso recorrerá los nodos de su fila en la base anterior. Comprobaremos el orden de las llamadas y sus argumentos. Si un nodo falla, las llamadas posteriores del recorrido normal no deberán ejecutarse; el control deberá seguir por el manejador. Los conteos C y R incluyen las llamadas intentadas, aunque una de ellas lance una excepción.

Revisión del cierre por el coordinador

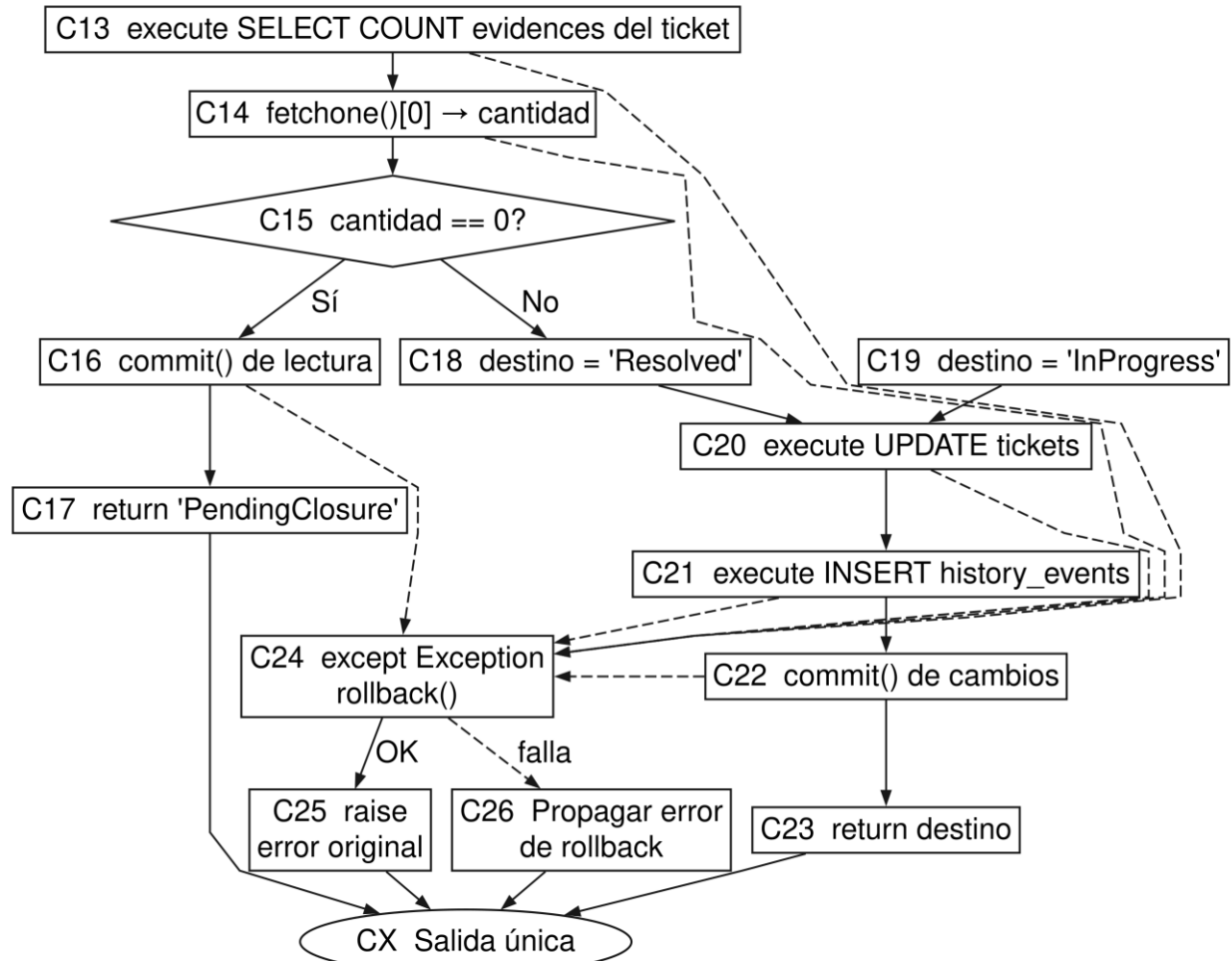
Grafo de revisar_cierre: panel A

A. Estado, acción y solución



Grafo de revisar_cierre: panel B

B. Evidencia, cambios y excepciones



$$N = 27; E = 42; P = 1. V(G) = 42 - 27 + 2 = 17.$$

Caminos independientes de revisión de cierre

Base de caminos y arista incorporada

Caso	Recorrido completo	Arista nueva
PE-20	C1 → C2 → C3 → C4 → C7 → C8 → C10 → C13 → C14 → C15 → C18 → C20 → C21 → C22 → C23 → CX	C1 → C2
PE-21	C1 → C2 → C3 → C4 → C5 → C6 → CX	C4 → C5
PE-22	C1 → C2 → C3 → C4 → C7 → C19 → C20 → C21 → C22 → C23 → CX	C7 → C19
PE-23	C1 → C2 → C3 → C4 → C7 → C8 → C10 → C11 → C12 → CX	C10 → C11
PE-24	C1 → C2 → C3 → C4 → C7 → C8 → C9 → C10 → C11 → C12 → CX	C8 → C9
PE-25	C1 → C2 → C3 → C4 → C7 → C8 → C10 → C13 → C14 → C15 → C16 → C17 → CX	C15 → C16
PE-26	C1 → C2 → C24 → C25 → CX	C2 → C24
PE-27	C1 → C2 → C3 → C24 → C25 → CX	C3 → C24
PE-28	C1 → C2 → C3 → C4 → C5 → C24 → C25 → CX	C5 → C24
PE-29	C1 → C2 → C3 → C4 → C7 → C8 → C10 → C11 → C24 → C25 → CX	C11 → C24
PE-30	C1 → C2 → C3 → C4 → C7 → C8 → C10 → C13 → C24 → C25 → CX	C13 → C24
PE-31	C1 → C2 → C3 → C4 → C7 → C8 → C10 → C13 → C14 → C24 → C25 → CX	C14 → C24
PE-32	C1 → C2 → C3 → C4 → C7 → C8 → C10 → C13 → C14 → C15 → C16 → C24 → C25 → CX	C16 → C24
PE-33	C1 → C2 → C3 → C4 → C7 → C8 → C10 → C13 → C14 → C15 → C18 → C20 → C24 → C25 → CX	C20 → C24
PE-34	C1 → C2 → C3 → C4 → C7 → C8 → C10 → C13 → C14 → C15 → C18 → C20 → C21 → C24 → C25 → CX	C21 → C24
PE-35	C1 → C2 → C3 → C4 → C7 → C8 → C10 → C13 → C14 → C15 → C18 → C20 → C21 → C22 → C24 → C25 → CX	C22 → C24
PE-36	C1 → C2 → C24 → C26 → CX	C24 → C26

Entradas y resultados internos esperados

Caso	Preparación o fallo provocado	Criterio verificable
PE-20	F-C; action="Approval"; solución con texto; COUNT=1.	return 'Resolved'. C=1; R=0. COUNT filtrado por 301; UPDATE con ("Resolved", 301); historial con (301, 7, "Approval").
PE-21	F-C; lectura ("InProgress", "Trabajo en curso").	return 'InProgress'. C=1; R=0. Retorna el estado consultado; commit de lectura; no COUNT, UPDATE ni INSERT.
PE-22	F-C; action="Rejection".	return 'InProgress'. C=1; R=0. UPDATE con ("InProgress", 301); historial con (301, 7, "Rejection"); no COUNT.
PE-23	F-C; lectura ("PendingClosure", "").	return 'PendingClosure'. C=1; R=0. No normaliza None; termina por strip(); commit de lectura; no COUNT ni escrituras.
PE-24	F-C; lectura ("PendingClosure", None).	return 'PendingClosure'. C=1; R=0. solution pasa de None a ""; termina por strip(); commit de lectura; no COUNT ni escrituras.
PE-25	F-C; solución con texto; COUNT=0.	return 'PendingClosure'. C=1; R=0. COUNT devuelve cero; commit de lectura; no UPDATE ni INSERT.
PE-26	F-C; error_original en C2.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-27	F-C; error_original en C3.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-28	Datos de PE-21; error_original en C5.	Propaga la misma instancia error_original. C=1; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-29	Datos de PE-23; error_original en C11.	Propaga la misma instancia error_original. C=1; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-30	F-C; error_original en C13.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-31	F-C; error_original en C14.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-32	Datos de PE-25; error_original en C16.	Propaga la misma instancia error_original. C=1; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.

Caso	Preparación o fallo provocado	Criterio verificable
PE-33	F-C; error_original en C20.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-34	F-C; error_original en C21.	Propaga la misma instancia error_original. C=0; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-35	F-C; error_original en C22 antes del efecto de commit.	Propaga la misma instancia error_original. C=1; R=1. No ejecuta las operaciones posteriores al punto fallido, salvo rollback y raise.
PE-36	F-C; error_original en C2 y error_rollback en C24.	Propaga error_rollback; __context__ is error_original. C=0; R=1. No llega al raise del manejador ni retorna éxito.

El caso recorrerá los nodos de su fila en la base anterior. Comprobaremos el orden de las llamadas y sus argumentos. Si un nodo falla, las llamadas posteriores del recorrido normal no deberían ejecutarse; el control deberá seguir por el manejador. Los conteos C y R incluyen las llamadas intentadas, aunque una de ellas lance una excepción.

Casos complementarios y criterios de aceptación

Las bases anteriores cubrirán cada nodo y cada arista de los tres grafos, incluidas las excepciones representadas. Para revisar el ciclo y las variantes que siguen un mismo camino usaremos además los casos siguientes. Los valores no crearán reglas adicionales de cantidad o tamaño de fotografías.

Variantes de condiciones, repetición del ciclo y flujo de datos

Caso	Datos	Comprobación interna
AD-01	description=" " y "\t\n", frente a texto con contenido.	Espacios o tabulaciones: return None en R3 sin SQL. Con contenido, el control alcanza R4.
AD-02	Tres evidencias distintas. Después, repetir provocando error en la segunda inserción.	Sin fallo, i avanza 0→1→2→3 y hay cinco INSERT en total. Con fallo intermedio se omiten la tercera evidencia, historial y commit; se intenta rollback y self.id conserva None.
AD-03	Cada uno de los seis tipos de falla del catálogo; estado Validated.	El UPDATE recibe el prioridad_id consultado y el retorno es el nombre correspondiente. High y Standard recorren la misma rama de P4.
AD-04	Lectura ("PendingClosure", " "); luego solución consultada con texto, COUNT=0 y COUNT=1.	Los espacios terminan en C12. Sin evidencia se retorna en C17. Con texto y evidencia se alcanza Resolved por C18.
AD-05	Una evidencia relacionada solamente con otro ticket.	La consulta COUNT incluye WHERE ticket_id = %s y recibe (self.id,). En integración, la fila de otro ticket no incrementa el conteo del revisado.

Criterios de aceptación de las pruebas

Aceptaremos un caso cuando coincidan el retorno o la instancia de excepción, la secuencia de llamadas y los parámetros del SQL con el recorrido previsto. En registro con n evidencias habrá $n + 2$ INSERT y un commit. Las guardas previas no producirán llamadas a la conexión ni al cursor. self.id se asignará después de confirmar, y los INSERT de evidencias e historial conservarán ticket_id.

En priorización y cierre, un retorno anticipado posterior a SELECT tendrá un commit de la transacción de lectura y ninguna escritura. Un cambio aceptado tendrá el UPDATE correspondiente, el evento de history_events y el commit. Cuando falle una llamada dentro de try, comprobaremos un intento de rollback y la ausencia de retorno de éxito.

Si rollback termina correctamente, la excepción propagada deberá ser la misma instancia recibida por except. Si también falla rollback, se propagará error_rollback y su __context__ conservará error_original. El fallo de commit tendrá C=1 porque la llamada se intentó. Los mocks comprobarán el flujo del código; la reversión efectiva de filas requerirá una prueba de integración.

Comprobaciones complementarias del SQL

En una base preparada para pruebas verificaremos code de 24 y 25 caracteres, un code repetido y un file_path repetido. Mantendremos válidas las demás referencias para distinguir cada restricción. También verificaremos que una falla de escritura anterior a la confirmación no deje un ticket, evidencia o historial parcial. Estas comprobaciones revisarán las restricciones y la transacción reales; no aumentarán V(G) porque no agregan decisiones al Python mostrado.

Herramientas y ejecución de las pruebas

Con pytest prepararemos los datos de cada condición y ejecutaremos el modelo de referencia. Usaremos casos separados para los retornos anticipados, el recorrido de evidencias, las respuestas de las consultas y las decisiones de cierre. Las aserciones comprobarán el retorno exacto, los parámetros SQL y la cantidad de escrituras. En un registro con n evidencias esperaremos $n + 2$ INSERT, un commit y ningún rollback. Al fallar una validación esperaremos None y ninguna escritura. Cada caso quedará relacionado con el recorrido que representa en el grafo.

Con los mocks de la conexión y del cursor provocaremos errores en puntos concretos. Guardaremos una instancia `error_simulado` y usaremos `pytest.raises(Exception)` as `exc` junto con `exc.value is error_simulado` para comprobar su identidad cuando rollback termine correctamente. Si falla commit, comprobaremos que se intentó una vez. Si rollback produce `error_rollback`, la excepción recibida deberá ser esa instancia y conservar `error_simulado` en `__context__`. También revisaremos que no se ejecuten escrituras posteriores al fallo ni se devuelva éxito. La reversión efectiva de filas se comprobará aparte, en una base preparada para las pruebas.

Con `coverage.py` registraremos las sentencias ejecutadas frente al total de sentencias medibles y las ramas recorridas frente al total de ramas medibles del modelo. Esos totales quedarán identificados en el informe; la matriz de aristas y los fallos inducidos completarán la revisión de los recorridos excepcionales. Con Pylint revisaremos nombres, variables, importaciones y estructura del Python ejecutable. Las llamadas de transacción se comprobarán con pytest. Los resultados del modelo se identificarán como tales; las mediciones de la aplicación Android y del acceso mediante la API quedarán pendientes de sus pruebas.

Prepararemos una fixture por escenario y configuraremos `side_effect` en la llamada exacta que queremos hacer fallar. Por ejemplo, en PE-08 el primer `execute` insertará el ticket, `fetchone` devolverá su id y el segundo `execute` lanzará la excepción al guardar el historial. Así podremos comprobar el paso hacia `rollback` sin utilizar la base de datos real.

Medición y revisión previstas

Uso de las herramientas sobre los archivos del modelo

Herramienta	Aplicación y evidencia
pytest	Ejecutar fixtures y casos; comprobar retornos, identidad de excepciones, argumentos de SQL y orden de llamadas. El ejemplo siguiente desarrolla el fallo de historial.
coverage.py	Ejecutar: <code>coverage run --branch -m pytest structural_ticket_tests.py</code> . Generar <code>coverage report -m</code> y <code>coverage html</code> . Revisar líneas y ramas faltantes de los tres métodos. Comparar las salidas excepcionales con la matriz del grafo, porque una cifra global no explica cada error provocado.
Pylint	Ejecutar sobre <code>structural_ticket_model.py</code> y el archivo de pruebas. Revisar variables sin usar, nombres no definidos, importaciones y estructura. Analizar el <code>except Exception</code> por su función de <code>rollback</code> y relanzamiento; no reemplazar esa revisión por una nota automática.

La aceptación de cobertura exigirá que las líneas ejecutables y ambas salidas de cada `if` o `while` de los tres métodos estén recorridas. También deberá existir un caso testigo por cada arista excepcional de las tablas. Informaremos el archivo medido, las líneas o ramas pendientes y el alcance de la ejecución. Una medición del modelo no se presentará como cobertura de la aplicación Android o de la API.

Conclusiones

En el registro, un error en el incremento de *i* o en el límite del ciclo podría repetir una evidencia u omitir la última. Omitir una evidencia dejaría información incompleta. Repetir el mismo *file_path* produciría un fallo por su restricción UNIQUE y llevaría el método a rollback. Esto puede dificultar la revisión de la falla y hacer necesaria otra comprobación o un desplazamiento adicional. Por eso revisaremos el recorrido completo y los identificadores enviados en cada inserción, además del valor que devuelve registrar.

La cobertura de la lógica de priorización del SGF-AP tiene una relación directa con la oportunidad de atención de las fallas reportadas. La Alcaldía de Medellín señala que utilizar información sobre las fallas para orientar técnicamente a las cuadrillas contribuye a agilizar las intervenciones, reducir desplazamientos innecesarios y disminuir costos operativos. A partir de este antecedente, identificamos una consecuencia posible para nuestro proyecto: si una condición incorrecta deja un reporte de alto riesgo almacenado, pero excluido de la información utilizada para organizar su atención, las decisiones de mantenimiento se tomarían con información incompleta. Esto podría llevar a atender casos de menor prioridad mientras continúa pendiente una situación que requiere atención preferencial.

En nuestro catálogo SQL, los cables expuestos y los postes con riesgo de caída corresponden a prioridad alta. Si el método omite la actualización de uno de estos reportes o utiliza una prioridad equivocada, podría retrasarse la intervención y prolongarse la exposición al peligro reportado. Cuando una falla también produce pérdida de iluminación, la demora prolongaría la deficiencia de visibilidad en el sector afectado. Esta consecuencia se relaciona con la finalidad del servicio descrita por EPM: “proporcionar condiciones adecuadas de visibilidad para la circulación peatonal y vehicular”. Por tanto, el análisis identifica un riesgo para la

atención y el uso seguro del espacio público, sin atribuir al error informático accidentes que no se han documentado.

El impacto financiero posible se relaciona con el uso de los recursos destinados a prestar el servicio. El Estudio Técnico de Referencia de Medellín para 2025–2028 establece que el alumbrado público se financia principalmente mediante impuestos a la ciudadanía y que los costos de administración, operación y mantenimiento incluyen la gestión de fallas, el mantenimiento preventivo y correctivo y la atención de peticiones, quejas y reclamos. En el escenario analizado, recuperar reportes omitidos, revisar su clasificación y reorganizar las intervenciones podría consumir tiempo del personal que estaría disponible para otras tareas de atención. Si esa reorganización ocasionara desplazamientos adicionales, también podría aumentar el gasto operativo. Estas son consecuencias posibles del fallo; calcular su valor exigiría conocer los tiempos efectivamente empleados, los desplazamientos realizados y sus costos.

Por estas razones, las pruebas de priorizar deben comprobar ambos resultados de la condición que evalúa el estado del ticket y verificar qué ocurre con los valores obtenidos de la consulta. Para un reporte validado cuyo tipo de falla corresponde a High, se debe verificar que el método utilice la prioridad asociada en la base de datos, ejecute la actualización del mismo ticket, registre la acción de priorización y retorne High. Una condición sin probar podría permitir una salida anticipada que interrumpiera ese recorrido. Relacionar la cobertura con estos resultados concretos permite evaluar si la lógica conserva la información necesaria para atender los casos de mayor riesgo y utilizar adecuadamente los recursos del mantenimiento.

En el cierre, una condición invertida puede permitir que el recorrido continúe sin solución o sin evidencia. También debemos revisar el fallo que ocurre después de actualizar el ticket y antes de guardar el historial. Si no comprobamos ese recorrido, podríamos conservar un cierre

que no coincida con el seguimiento disponible del caso. Las pruebas revisarán las decisiones, la confirmación y la recuperación de los datos. La suficiencia de la reparación seguirá dependiendo de la revisión del coordinador; el modelo comprobará que su acción produzca el cambio previsto sobre el mismo ticket. Para Medellín, corregir tarde estos errores puede aumentar las horas de coordinación, los desplazamientos y el costo de atención de la falla.